Learning Assistant Engine Architecture

> *Prepared for: Investor Tech Audit Workshop Date: 2026-03-29 | Version: 1.0 Source: `src/engine/learning-assistant/` (19 files)*

---

1. Design Philosophy

The Learning Assistant Engine is designed as a **domain-agnostic, SaaS-extractable** adaptive learning system. While currently deployed for body language coaching, its core interfaces make no assumptions about the learning domain. The engine could power a piano practice app, a language learning tool, or a coding bootcamp with only the content provider and skill categories swapped.

Core Principles

1. **Interface-driven**: All components communicate through TypeScript interfaces (`ILearner` , `IContentProvider` , `INotificationChannel`), not concrete implementations.
2. **Registry pattern**: Channels, analyzers, and planners are registered at runtime, enabling plug-and-play extension.
3. **Singleton engine**: One `LearningAssistantEngine` instance per server process, instantiated lazily via `getEngine()` .
4. **DB-based queue**: Notifications use a PostgreSQL queue (not Redis/SQS), keeping infrastructure minimal while supporting retries and audit trails.
5. **Timezone-aware**: All scheduling respects the user's timezone, processing users in hourly brackets.

---

2. Module Structure

```
src/engine/learning-assistant/
  index.ts                  -- Barrel exports + singleton + channel auto-registration
  core/
    types.ts                -- All interfaces and type definitions
    config.ts               -- Default config + timezone utilities
    engine.ts               -- LearningAssistantEngine class
    registry.ts             -- Singleton registry for channels
  analyzers/
    progress-analyzer.ts    -- 7-day learning progress snapshot
    engagement-analyzer.ts  -- 30-day engagement scoring (0-100)
    skill-gap-analyzer.ts   -- Weak/strong/neglected skill identification
  planners/
    activity-planner.ts     -- Daily plan generation (3-5 activities)
    reminder-planner.ts     -- Schedule notifications based on context
    motivation-planner.ts   -- Template + GPT-4o-mini message generation
  channels/
    channel-interface.ts    -- BaseNotificationChannel abstract class
    in-app-channel.ts       -- Database-stored in-app notifications
    push-channel.ts         -- Web Push (VAPID) notifications
    email-channel.ts        -- Resend email with branded HTML templates
    whatsapp-channel.ts     -- Twilio WhatsApp messaging
  templates/
    template-engine.ts      -- Mustache-style {{variable}} renderer
```

```
    email-templates.ts      -- Rich HTML email templates (weekly report)
  scheduler/
    scheduler.ts            -- Queue management: schedule + process
```

---

3. Core Interfaces

ILearner

```
interface ILearner {
  userId: string
  timezone: string                              // IANA timezone
  preferredChannels: string[]                   // ['in_app', 'push', 'email']
  optOutChannels: string[]                      // User-disabled channels
  optimalPracticeTime: string | null            // "09:00" (learned from behavior)
  practiceTimeConfidence: number                // 0-1 confidence score
  avgSessionsPerWeek: number
  engagementScore: number                       // 0-100
  weakSkills: string[]                          // ['eye_contact', 'posture']
  strongSkills: string[]
  notificationFrequency: 'quiet' | 'normal' | 'active'
}
```

IContentProvider

```
interface IContentProvider {
  getNextLessons(userId: string, limit: number): Promise<IContentItem[]>
  getSkillPractice(userId: string, skill: string): Promise<IContentItem[]>
  getArcadeChallenges(userId: string, limit: number): Promise<IContentItem[]>
}
```

INotificationChannel

```
interface INotificationChannel {
  readonly name: string
  send(userId: string, payload: NotificationPayload): Promise<DeliveryResult>
  isAvailable(userId: string): Promise<boolean>
}
```

NotificationPayload

```
interface NotificationPayload {
  triggerType: TriggerType     // 12 trigger types
  title: string
  body: string
  deepLink?: string            // In-app navigation target
  priority: 'low' | 'normal' | 'high'
```

```
  metadata?: Record<string, unknown>
}
```

Trigger Types (12)

Trigger	When Fired	Priority
`morning_motivation`	Morning cycle, low engagement users	normal
`streak_warning`	Evening cycle, active streak not continued today	high
`streak_broken`	After streak resets to 0	normal
`comeback`	3+ days inactive	high
`level_up`	On activity (XP threshold crossed)	high
`badge_earned`	On activity (badge criteria met)	high
`leaderboard_change`	Rank change detected	normal
`skill_gap_nudge`	Morning cycle, neglected skill 14+ days	low
`weekly_report`	Monday morning cycle	normal
`social_nudge`	Following user achieves milestone	low
`celebration`	Streak milestone (7, 14, 21... days)	normal
`new_content`	New clips/courses added	low

---

4. Engine Lifecycle

4.1 Morning Cycle (Cron: 08:00 UTC daily)

```
Vercel Cron (/api/cron/morning)
    |
    v
engine.runMorningCycle()
    |
    +-- getTimezonesBracket(8)          // Find timezones where local time = 8 AM
    |   Returns: ['America/New_York']   // (example at 13:00 UTC)
    |
    +-- Query LearnerProfile WHERE timezone IN bracket
    |   Also: Users without profile (default UTC)
    |
    +-- For each user:
        |
        +-- buildLearnerContext(userId)
        |   |
        |   +-- getOrCreateLearnerProfile()
        |   +-- analyzeProgress()        // 7-day snapshot
        |   +-- analyzeEngagement()      // 30-day score
```

```
        |   +-- analyzeSkillGaps()       // Weak/strong/neglected
        |   |
        |   Returns: LearnerContext { learner, progress, engagement, skillGaps }
        |
        +-- generateDailyPlan(userId, ctx)
        |   |
        |   +-- 1. Skill gap lesson (priority 1)
        |   +-- 2. Next lesson in path (priority 2)
        |   +-- 3. Arcade challenge (priority 3)
        |   +-- 4. Neglected skill review (priority 4)
        |   +-- 5. Daily quest prompt (priority 5)
        |   |
        |   Upsert to LearningPlan table (userId + 'daily' + date)
        |
        +-- scheduleReminders(userId, ctx)
        |   |
        |   +-- Morning motivation (if engagement < 80)
        |   +-- Streak warning (if inactive 1+ day)
        |   +-- Comeback message (if inactive 3+ days)
        |   +-- Skill gap nudge (if neglected skills exist)
        |   |
        |   Writes ScheduledNotification records
        |
        +-- updateLearnerProfile(userId, ctx)
            |
            Persists: engagementScore, optimalPracticeTime,
                      weakSkills, strongSkills
```

4.2 Activity Events (Real-time)

```
User completes activity (lesson, arcade, practice, etc.)
    |
    v
gamification.processActivity(userId, type)
    |
    +-- awardXp()
    +-- checkAndUpdateStreak()
    +-- updateQuestProgress()
    +-- evaluateBadges()
    |
    v
engine.onActivity(userId, result)    // Fire-and-forget
    |
    +-- If leveledUp: schedule 'level_up' notification
    +-- If badgesEarned: schedule 'badge_earned' for each
    +-- If streak % 7 === 0: schedule 'celebration'
    +-- Update engagement score asynchronously
```

4.3 Evening Cycle (Cron: 22:00 UTC daily)

```
Vercel Cron (/api/cron/engagement-check)
    |
    v
engine.runEveningCycle()
    |
    +-- getTimezonesBracket(20)         // Timezones where local = 8 PM
    +-- Query users with active streaks
    +-- If streak > 0 AND lastActivityDate != today:
    |   Schedule 'streak_warning' (immediate, high priority)
    |
    v
engine.processNotificationQueue()
    |
    +-- processQueue(batchSize=50)
        |
        +-- Query ScheduledNotification WHERE status='pending'
        |   AND scheduledFor <= now AND attempts < 3
        |   ORDER BY priority ASC, scheduledFor ASC
        |
        +-- For each notification:
            +-- Resolve channel from registry
            +-- Generate title/body (template or GPT-4o-mini)
            +-- channel.send(userId, payload)
            +-- On success: status='sent', log to NotificationLog
            +-- On failure: increment attempts, retry later
            +-- On max retries: status='failed'
```

4.4 Weekly Report (Monday morning)

```
engine.runWeeklyReport()
    |
    +-- Batch users (50 at a time, cursor pagination)
    +-- For each user:
        +-- analyzeProgress() + analyzeEngagement()
        +-- Gather: badges, quests, XP, skill gaps
        +-- Determine top achievement + next week focus
        +-- Schedule in-app notification (summary)
        +-- Generate rich HTML email (weeklyReportHtml)
        +-- Schedule email notification with HTML body
```

---

5. Analyzer Pipeline

Progress Analyzer

Metric	Calculation	Window
`lessonsThisWeek`	COUNT FoundationProgress WHERE completedAt >= 7 days ago	7 days
`avgQuizScore`	AVG of quizScore WHERE quizPassed = true	7 days
`arcadeScores.avg`	AVG of ArcadeAttempt.score	7 days

`arcadeScores.count`	COUNT of ArcadeAttempt	7 days
`questCompletionRate`	Completed quests / total quests	7 days
`xpVelocity`	Total XP / 7 (XP per day)	7 days

Engagement Analyzer

Computes a 0-100 composite engagement score:

Factor	Weight	Calculation
Recency	0-30	30 if active today, 25 if yesterday, 15 if 1-3 days, 5 if 3-7 days, 0 if 7+ days
Frequency	0-30	activities_per_week * 5, capped at 30
Streak	0-20	currentStreak * 2, capped at 20
Consistency	0-20	(unique_active_days / 14) * 20, over last 14 days

Also determines:

- **Optimal practice time**: Peak hour from activity timestamps (requires 5+ activities)
- **Practice time confidence**: Peak hour frequency / total activities * 2 (capped at 1.0)
- **isDropping**: True if last-week activities < 50% of previous week

Skill Gap Analyzer

Classifies each of 8 skill categories:

Classification	Criteria
Weak	Average of last 5 scores < 50
Strong	Average of last 5 scores >= 75
Neglected	Has baseline but 0 practice, OR not practiced in 14+ days

Skill categories: `eye_contact`, `facial_expressions`, `gestures`, `posture`, `vocal_tone`, `vocal_pacing`, `spatial_awareness`, `mirroring`

---

6. Planner Pipeline

Activity Planner

Generates 3-5 personalized activities per day:

Priority	Activity Type	Selection Logic
1	Skill gap lesson	First uncompleted lesson in weakest skill's course
2	Next lesson	First uncompleted lesson across all courses (ordered)
3	Arcade challenge	Challenge not yet attempted by user

4	Neglected skill review	Lesson in most-neglected skill category
5	Daily quest prompt	Generic prompt to complete quests

Reminder Planner

Schedules 0-4 notifications per user per morning cycle:

Notification	Condition	Timing	Priority
Morning motivation	engagement < 80	User's optimal practice time	normal
Streak warning	inactive 1+ day AND engagement > 20	Evening (20:00 local)	high
Comeback	inactive 3+ days	Optimal practice time	high
Skill gap nudge	has neglected skills AND engagement > 30	Practice time + 2 hours	low

Motivation Planner

Three-tier message generation:

```
1. NotificationTemplate table lookup (random selection for variety)
   |
   v (if no template found)
2. GPT-4o-mini generation (model: gpt-4o-mini, max 150 tokens)
   |
   v (if API fails)
3. Hardcoded fallback messages (5 defaults)
```

---

7. Channel Architecture

Channel Registration (Auto at Import)

```
// src/engine/learning-assistant/index.ts
function initChannels() {
  const registry = getRegistry()
  registry.registerChannel(new InAppChannel())                    // Always

  if (process.env.VAPID_PUBLIC_KEY && process.env.VAPID_PRIVATE_KEY)
    registry.registerChannel(new PushChannel())                   // If VAPID configured

  if (process.env.RESEND_API_KEY)
    registry.registerChannel(new EmailChannel())                  // If Resend configured

  if (process.env.TWILIO_ACCOUNT_SID)
    registry.registerChannel(new WhatsAppChannel())               // If Twilio configured
}
```

Channel Implementations

Channel	Transport	Library	Availability Check
in_app	Database write (ScheduledNotification + NotificationLog)	Prisma	Always available
push	Web Push Protocol (VAPID)	`web-push` npm	PushSubscription exists for user
email	SMTP via Resend API	`resend` npm	User has email AND not opted out
whatsapp	Twilio WhatsApp API	`twilio` npm	LearnerProfile.whatsappOptIn = true AND phone set

Channel: Push Notifications Detail

- Uses Web Push standard (VAPID authentication)
- Manages PushSubscription records in database
- Sends to all subscribed devices per user
- Auto-cleans expired subscriptions (410/404 responses)
- Supports action buttons per trigger type:
  - `streak_warning` -> "Practice now"
  - `morning_motivation` -> "Start learning"
  - `skill_gap_nudge` -> "Practice skill"

Channel: Email Detail

- Branded HTML email from "Coach Ney coach@seeneyu.com"
- Two template modes:
  1. Rich HTML (weekly report with stats, skill breakdown, achievements)
  2. Simple branded wrapper (title + body in dark-theme email template)
- Respects user opt-out via LearnerProfile.optOutChannels

Channel: WhatsApp Detail

- Twilio messaging API
- Requires explicit opt-in (LearnerProfile.whatsappOptIn)
- Sends plain text (WhatsApp doesn't support HTML)
- Messages from configurable `TWILIO_WHATSAPP_FROM` number

---

8. Template System

Variable Interpolation

```
// Mustache-style: {{variable}}
renderTemplate("Hi {{firstName}}, you have a {{streak}}-day streak!", {
  firstName: "Jane",
  streak: 7
```

```
})
// -> "Hi Jane, you have a 7-day streak!"
```

Standard Variables (Auto-Resolved)

Variable	Source	Example
`{{name}}`	User.name	"Jane Doe"
`{{firstName}}`	First word of User.name	"Jane"
`{{level}}`	UserGamification.level	5
`{{xp}}`	UserGamification.totalXp	1250
`{{streak}}`	UserGamification.currentStreak	7
`{{longestStreak}}`	UserGamification.longestStreak	14
`{{triggerType}}`	Current trigger	"morning_motivation"
`{{lessonsThisWeek}}`	Weekly report context	3
`{{xpThisWeek}}`	Weekly report context	350

Template Storage

Templates are stored in the `NotificationTemplate` table with:

- `slug` (unique identifier)
- `triggerType` + `channel` + `locale` (unique combination)
- `title` and `body` with `{{variable}}` placeholders
- `variables` JSON array documenting available variables
- `isActive` flag for A/B testing

---

9. Scheduler: Vercel Cron + DB Queue

Architecture

```
Vercel Cron (2 jobs)
    |
    +-- 08:00 UTC -> /api/cron/morning
    |   Calls: engine.runMorningCycle()    [analyze + plan + schedule]
    |   Calls: engine.runWeeklyReport()    [if Monday]
    |
    +-- 22:00 UTC -> /api/cron/engagement-check
        Calls: engine.runEveningCycle()    [streak warnings]
        Calls: engine.processNotificationQueue()  [deliver pending]
```

Queue Processing

Parameter	Value	Configurable

Batch size	50 per invocation	`config.batchSize`
Max retries	3	`config.maxRetries`
Retry delay	15 minutes	`config.retryDelayMinutes`
Priority ordering	high > normal > low (alphabetical)	N/A
Status lifecycle	pending -> sent / failed	N/A

Timezone Bracketing

The engine processes users in 1-hour timezone brackets. When the cron fires at UTC hour H, it finds all IANA timezones where the local hour matches the target:

```
// At 13:00 UTC, morningCycleHour=8:
// 13 + (-5) = 8 -> America/New_York matches
// 13 + (-6) = 7 -> America/Chicago does NOT match
```

This ensures users receive morning notifications at their local 8 AM, not all at UTC 8 AM.

---

10. SaaS Extraction Strategy

The Learning Assistant Engine is designed for extraction as a standalone npm package. Here is the extraction path:

Step 1: Interface Extraction

The `core/types.ts` file already contains all domain-agnostic interfaces. No Seeneyu-specific types leak into the core.

Step 2: Adapter Pattern

```
// Current: Seeneyu-specific content provider
const lesson = await prisma.foundationLesson.findFirst(...)

// Extracted: Injected content provider
const provider: IContentProvider = config.contentProvider
const lessons = await provider.getNextLessons(userId, 3)
```

Step 3: Configuration for Different Domains

```
// Body language coaching (current)
const engine = new LearningAssistantEngine({
  skillCategories: ['eye_contact', 'gestures', 'posture', ...],
  morningCycleHour: 8,
  comebackThresholdDays: 3,
  gptModel: 'gpt-4o-mini',
})

// Piano practice (hypothetical)
```

```
const engine = new LearningAssistantEngine({
  skillCategories: ['scales', 'chords', 'sight_reading', 'dynamics'],
  morningCycleHour: 7,
  comebackThresholdDays: 2,
  gptModel: 'gpt-4o-mini',
})

// Language learning (hypothetical)
const engine = new LearningAssistantEngine({
  skillCategories: ['vocabulary', 'grammar', 'listening', 'speaking'],
  morningCycleHour: 8,
  comebackThresholdDays: 5,
  gptModel: 'gpt-4o-mini',
})
```

Step 4: Database Migration

The 5 engine-specific models (`LearnerProfile` , `LearningPlan` , `ScheduledNotification` , `NotificationLog` , `NotificationTemplate`) can be extracted to a separate schema or provided via a database adapter interface.

Effort Estimate

Task	Effort
Extract interfaces + core engine	2 days
Build database adapter	1 day
Build content provider adapter	1 day
npm package setup + tests	2 days
Documentation	1 day
Total	**~1 week**

---

11. Configuration Reference

```
interface EngineConfig {
  morningCycleHour: number       // Default: 8
  eveningCycleHour: number       // Default: 20
  maxNotificationsPerDay: number // Default: 5
  minEngagementForNudge: number  // Default: 80
  streakWarningHours: number     // Default: 4
  comebackThresholdDays: number  // Default: 3
  weeklyReportDay: number        // Default: 1 (Monday)
  batchSize: number              // Default: 50
  maxRetries: number             // Default: 3
  retryDelayMinutes: number      // Default: 15
```

```
  gptModel: string              // Default: 'gpt-4o-mini'
}
```

---

12. Limitations and Honest Assessment

Area	Current State	Improvement Path
Queue throughput	DB-based, 50/batch	Migrate to Redis or SQS at 10K+ users
Timezone accuracy	Approximate (static offset table)	Use `luxon` or `date-fns-tz` for DST handling
Content provider	Hardcoded Prisma queries	Implement IContentProvider interface
A/B testing	Template randomization only	Add proper experiment framework
Analytics	NotificationLog for audit	Add delivery rate dashboards
Multi-language	English only (locale field exists)	Template translation pipeline
Channel fallback	No automatic fallback	Add waterfall: push -> email -> whatsapp

---

Document prepared by Data Engineer role. All code references verified against `src/engine/learning-assistant/` (19 files) as of 2026-03-29.